COS314 ASSIGNMENT 2
COMPARING EFFECTIVENESS OF META-HEURISTICS TO SOLVE PROBLEM INSTANCES

Aundrea N. Ncube - 22747363
Amantle K. Temo - 23539764
David E. Kalu - 23534975

1.Genetic Algorithm

(a) Functionality:

Overview:
Our GA implementation follows a population-based metaheuristic approach to tackle the 0/1 Knapsack problem.
A fixed_size population of candidate solutions is maintained across generations, with each generation applying tournament selection, single-point crossover, and bit-filp mutation to drive the search toward better solutions.
Representation:
Each solution is represented as a binary array where a value of true at indexed at i, shows that i has been selected as per the standard binary encoding for knapsack problem.

```
public boolean[] selected;
public void flipItem(int i, KnapsackInstance instance)
{
    if (selected[i])
        {
        // Remove item
        totalValue -= instance.values[i];
        totalWeight -= instance.weights[i];
    } else {
        // Add item
        totalValue += instance.values[i];
        totalWeight += instance.weights[i];
    }
    selected[i] = !selected[i];
}
```

Population Initialisation:
The initial population consisting of 100 individuals is created using the hybrid strategy.
The first individuals are constructed greedily by sorting items in descending order of value-to-weight ratio and selecting items until the capacity is exceeded.
This seeds the population with what we would say a great starting point. The remaining 99 individuals are generated randomly using **rand_gene.nextBoolean()** for each item bit. Any infeasible individual (overweight) is repaired before being added to the population.

```java
private Solution[] init(KnapsackInstance inst, Random rand_gene)
{
    Solution[] pop = new Solution[pop_size];

    pop[0] = greedyInit(inst);
    //random
    for (int s = 1; s < pop_size; s++)
    {
        Solution solu = new Solution(inst.numItems);
        for (int i = 0; i < inst.numItems; i++)
        {
            solu.selected[i] = rand_gene.nextBoolean();
        }

        solu.recalculateTotals(inst);

        if (!solu.isValid(inst))
        {
            repair(solu, inst, rand_gene);
        }
        pop[s] = solu;
    }

    return pop;
}
```

```java
private Solution greedyInit(KnapsackInstance inst)
{
    Integer[] order = new Integer[inst.numItems];
    for (int i = 0; i < inst.numItems; i++) order[i] = i;

    java.util.Arrays.sort(order, (a, b) -> {
        double ra = inst.weights[a] == 0 ? Double.MAX_VALUE : (double) inst.values[a] / inst.weights[a];
        double rb = inst.weights[b] == 0 ? Double.MAX_VALUE : (double) inst.values[b] / inst.weights[b];
        return Double.compare(rb, ra);
    });

    Solution sol = new Solution(inst.numItems);
    for (int i : order)
    {
        if (sol.totalWeight + inst.weights[i] <= inst.capacity)
        {
            sol.flipItem(i, inst);
        }
    }
    return sol;
}
```

Fitness Function:

A solution's fitness is measured by its total packed value,provided the combined weight of selected items does not exceed the knapsack capacity. Any solution that violates the weight constraint is assigned a fitness of zero, ensuring infeasible solutions are never favoured during selection.

s, ensuring infeasible solutions are never selected as parents.

```
public boolean isValid(KnapsackInstance instance)
{
    boolean valid;

    if(totalWeight <= instance.capacity)
    {
        valid = true;
    }
    else
    {
        valid = false;
    }
    return valid;
}

public int fitness(KnapsackInstance instance)
{
    int fitness;

    if(isValid(instance) == true)
    {
        fitness = totalValue;
    }
    else
    {
        fitness = 0;
    }

    return fitness;
}
```

Selection:

Parent selection uses tournament selection with a tournament size of 3. At each selection event, 3 individuals are drawn at random from the current population and the fittest among them is chosen as a parent. The process runs twice per reproduction cycle to obtain two parents, striking a balance between selection pressure and population diversity.

```
public Solution select(Solution[] pop, int tour_size, KnapsackInstance inst, Random rand_gene)
{
    Solution best = null;
    int bestFit = Integer.MIN_VALUE;

    for (int i = 0; i < tour_size; i++)
    {
        int idx = rand_gene.nextInt(pop.length);
        Solution candidate = pop[idx];
        int canidateFit = candidate.fitness(inst);

        if (best == null || canidateFit > bestFit)
        {
            best = candidate;
            bestFit = canidateFit;
        }
    }

    return best;
}
```

Crossover:
Single-point crossover is applied at a rate of pc = 0.8. A cut point is randomly selected within the chromosome bounds [1, n-1], splitting each parent into two segments. The first child inherits the left segment from parent 1 and the right segment from parent 2, while the second child receives the opposite combination. When the crossover rate is not triggered, both parents pass directly to the next generation without modification.

```
public Solution[] crossover(Solution p1, Solution p2, double pc, KnapsackInstance inst, Random rand_gene)
{
    /// not enough items to perform c_o
    if (inst.numItems <= 1)
    {
        return new Solution[]{p1.copy(), p2.copy()};
    }

    // skip c_o if rand draw >= pc
    if (rand_gene.nextDouble() >= pc)
    {
        return new Solution[]{p1.copy(), p2.copy()};
    }

    //choose crossover point in range 1 -> numItems-1
    int cp = 1 + rand_gene.nextInt(inst.numItems - 1);

    Solution offspring1 = new Solution(inst.numItems);
    Solution offspring2 = new Solution(inst.numItems);

    for (int i = 0; i < inst.numItems; i++)
    {
        offspring1.selected[i] = (i < cp) ? p1.selected[i] : p2.selected[i];
        offspring2.selected[i] = (i < cp) ? p2.selected[i] : p1.selected[i];
    }

    offspring1.recalculateTotals(inst);
    offspring2.recalculateTotals(inst);

    return new Solution[]{offspring1, offspring2};
}
```

Mutation:
Each offspring undergoes bit-flip mutation where every gene is independently flipped with probability pm = 1/n, giving an expected mutation of one bit per individual, maintaining diversity and preventing premature convergence.

```
public void mutate(Solution solu, double pm, KnapsackInstance inst, Random rand_gene)
{
    for (int i = 0; i < inst.numItems; i++)
    {
        if (rand_gene.nextDouble() < pm)
        {
            solu.flipItem(i, inst); // flipItem updates totals incrementally*
        }
    }
}
```

Repair Mechanism:
After mutation, any infeasible offspring is repaired using a randomised removal strategy, in our case the Fisher-Yates shuffle on selected items, then removing items one by one until feasibility is restored. This ensures all individuals in the population are always feasible.

```
private void repair(Solution solu, KnapsackInstance inst, Random rand_gene)
{

    ArrayList<Integer> idxSelected = new ArrayList<>();
    for (int i = 0; i < inst.numItems; i++)
    {
        if (solu.selected[i])
        {
            idxSelected.add(i);
        }
    }

    //// this is whats called the Fisher-Yates shuffle *
    for (int i = idxSelected.size() - 1; i > 0; i--)
    {
        int j = rand_gene.nextInt(i + 1);
        int tmp = idxSelected.get(i);
        idxSelected.set(i, idxSelected.get(j));
        idxSelected.set(j, tmp);
    }

    // lastly we deselect items until feasible
    for (int idx : idxSelected)
    {
        if (solu.isValid(inst))
        {
            break;
        }
        solu.flipItem(idx, inst);
    }

}
```

Elitism:
To prevent losing good solutions, the fittest individual from each generation is carried forward unchanged into the next, with an elite size of 1. A separate global best is also tracked throughout the entire run and returned as the final answer.

```java
public Solution run(KnapsackInstance inst, Random rand_gene)
{
    double pm = 1.0 / inst.numItems;
    Selection slctr = new Selection();
    Crossover crossover = new Crossover();
    Mutation mut = new Mutation();


    Solution[] pop = init(inst, rand_gene);
    Solution globalBest = findBest(pop, inst).copy();

    for (int gen = 0; gen < max_gen; gen++)
    {
        Solution best = findBest(pop, inst).copy();
        if (best.fitness(inst) > globalBest.fitness(inst))
        {
            globalBest = best.copy();
        }

        Solution[] nextGen = new Solution[pop_size];
        nextGen[0] = best;

        int i = 1;
        while (i < pop_size)
        {
            Solution p1 = slctr.select(pop, tour_size, inst, rand_gene);
            Solution p2 = slctr.select(pop, tour_size, inst, rand_gene);
            Solution[] offspring = crossover.crossover(p1, p2, pc, inst, rand_gene);

            for (Solution child : offspring)
            {
                mut.mutate(child, pm, inst, rand_gene);

                if (!child.isValid(inst))
                {
                    repair(child, inst, rand_gene);
                }

                if (i < pop_size)
                {
                    nextGen[i++] = child;
                }
            }
        }
```

```java
        pop = nextGen;
    }



    Solution lastBest = findBest(pop, inst);

    if (lastBest.fitness(inst) > globalBest.fitness(inst))
    {
        globalBest = lastBest.copy();
    }


    return globalBest;
}
```

(b) Environment & Setup:

The GA was setup with the following parameters:

Parameter	Value	Reason
Population size	100	Large enough for diversity, small enough for speed
Max generations	500	Sufficient convergence budget for all instances
Tournament size	3	Moderate selection pressure
Crossover probability (pc)	0.8	Standard value; high recombination rate
Mutation probability (pm)	1/n	Scales with problem size; expected 1 flip per individual
Elite size	1	Preserves best solution across generations

```java
private static void loadInstances(int seed)
{
    try
    {
        // first load the instances
        KnapsackInstance[] instances = KnapsackReader.readAll("data/");
        System.out.println("Loaded " + instances.length + " instances\n");

        // then prepare results storage
        List<String[]> results = new ArrayList<>();
        results.add(new String[]{"Instance", "Algorithm", "Seed", "BestValue", "KnownOptimum", "Runtime(s)"});

        //then run the alogrithms
        for (KnapsackInstance instance : instances)
        {
            System.out.println("Processing: " + instance.name);
            System.out.println("Capacity: " + instance.capacity + ", Items: " + instance.numItems);

            //Genetics Algorithm here
            Random gaRandom = new Random(seed);
            long gaStart = System.nanoTime();
            Solution gaSolution = ga(instance, gaRandom);
            long gaEnd = System.nanoTime();
            long gaRuntime = (gaEnd - gaStart) / 1_000_000;
            double gaRuntimeSec = gaRuntime / 1000.0;
```

```java
    private static final int    pop_size = 100,  max_gen = 500, tour_size = 3;
    private static final double  pc = 0.8;


//////////{::}\\\\\\\\\\\\\
    public Solution run(KnapsackInstance inst, Random rand_gene)
    {
        double pm = 1.0 / inst.numItems;
```

2. Iterated Local Search

(a) Functionality

Overview

Iterated Local Search is a metaheuristic that is trajectory-based. It improves a solution by repeating the local search and through perturbation of that single solution. Unlike the Genetic Algorithm which is a population-based method, ILS keeps one candidate solution at a given time and navigates the search space by moving between local optima instead of evolving a population.

Basic Pseudocode of ILS

```
s ← ConstructInitialSolution()
l ← LocalSearch(s)
    loop until stopping condition:
 p ← Perturbation(l)
 f ← LocalSearch(p)
    if fitness_function(f) > fitness_function(l) then l ← f
return l
```

Building the Initial Solution

ILS generates a starting solution through a greedy heuristic. Each item is given a value-to-weight ratio, a small seeded random noise is added to each ratio - up to 1%, before sorting. Items are then greedily added in descending order until the knapsack is full. The purpose of the random noise is to ensure different seed values produce different starting solutions to ensure diversity across each run. This is useful when testing the algorithm to collect statistics. ***(iteratedLocalSearch.java - greedyConstruct())***

```java
    ratio[i] = base + base * 0.01 * rng.nextDouble(); // add small random noise
}

java.util.Arrays.sort(order, (a, b) -> Double.compare(ratio[b], ratio[a]));

Solution sol = new Solution(num);
for (int i : order) {
    if (sol.totalWeight + inst.weights[i] <= inst.capacity) {
        sol.flipItem(i, inst);
    }
}
```

Local Search

The LocalSearch.java performs a local search procedure which improves a given solution by iteratively exploring two neighbourhoods until no further improvement is found, this is until a local optimum is found.

1-opt(Bit-flip)

For each item, the algorithm switches the item's inclusion status (adds - if excluded, removes, if included). If the end solution is valid as well as improving the total end result then that solution will be recorded as a candidate. After each pass, the the best flip is applied *(LocalSearch.java)*

```
for(int i = 0; i< inst.numItems; i++){
    boolean wasSelected = sol.selected[i];
    sol.flipItem(i, inst);
    if(sol.isValid(inst)){
        int gain = wasSelected ? -inst.values[i] : inst.values[i];

        if(gain > bestOne){
            bestOne = gain;
            bestFlip = i;
            bestIN =  -1;
            bestOUT =  -1;
        }
    }

    sol.flipItem(i, inst); //flip back
}
```

2-opt(item swap)

For each of the item pairs where one is included and the other excluded, the algorithm will remove the included item and add the excluded item simultaneously, if the swap is valid and improves the value, this is recorded as a candidate.
The 2-opt neighborhood is essential because certain optimal solutions cannot be reached by the 1-opt move alone, e.g. the instance `f4_l-d_kp_4_11`, the optimal solution requires including two specific items that a greedy approach won't naturally pick. The 2-opt resolves this by allowing a simultaneous exchange. *(LocalSearch.java)*

```java
for(int out=0; out<inst.numItems; out++){
    if(!sol.selected[out]) continue;

    for(int in = 0; in<inst.numItems; in++){
        if(sol.selected[in]) continue;

        sol.flipItem(out, inst);    //remove swap then add in
        sol.flipItem(in, inst);

        if(sol.isValid(inst)){
            int gain = inst.values[in] - inst.values[out];
            if(gain > bestOne){
                bestOne = gain;
                bestFlip = -1;
                bestIN = in;
                bestOUT = out;
            }
        }
        sol.flipItem(in, inst);    //flip back
        sol.flipItem(out, inst);
    }
}
```

The Repair

If the solution is over the weight limit i.e. not feasible, it is repaired before the search begins by removing one at a time, starting with whichever item gives the least value per unit of weight. This is done until feasibility is restored to ensure the solution is preserved. *(LocalSearch.java - repair())*

```java
while(!s.isValid(inst))
{
    double worstRatio = Double.MAX_VALUE;
    int worstIdx = -1;

    for(int i =0; i< inst.numItems; i++){
        if(s.selected[i]){
            double w = inst.weights[i];
            double ratio = (w==0) ? Double.MAX_VALUE : (double) inst.values[i] / w;
            if(ratio < worstRatio){
                worstRatio = ratio;
                worstIdx = i;
            }
        }
    }
    if(worstIdx == -1) break; //should not necessarily happen, but just in case
    s.flipItem(worstIdx, inst);
}
```

Local Search termination
Local Search stops when a full pass through 1-opt and 2-opt doesn't generate an improvement. At this stage, the solution is a local optimum. *(LocalSearch.java)*

```
if(bestFlip != -1){
    sol.flipItem(bestFlip, inst);
    improved = true;
}
else if(bestIN != -1){
    sol.flipItem(bestOUT, inst);
    sol.flipItem(bestIN, inst);
    improved = true;
}
```

Perturbation
When local search converges to local optimum, the perturbation `Perturbation.java` step shakes the solution before another local search pass is applied. This perturbation is large enough that local search cannot undo it but small enough so the resulting solution can retain or maintain useful information from the local optimum.

The perturbation operates as follows:
- Currently included and excluded items are collected into separate lists.
- Each list is shuffled using the Random instance:

```
Collections.shuffle(included, rng);
Collections.shuffle(excluded, rng);
```

- 2 randomly selected included items are removed from the solution and 2 randomly selected excluded items are added to the solution.
- If the result is invalid, the repair from LocalSearch is applied

The strength k=4 was chosen to provide sufficient disruption to escape local optima while keeping the solution to allow local search to recover. k=2 is too weak on the larger instances as it provides insufficient disruption to escape local optima, while k=6 may increase runtime without much improvement on the instances.

Acceptance Criteria
After perturbation and local search, the new solution is only kept if it is better than the current best, if not the algorithm will continue perturbing the best solution it has already found until a global best is found. This acceptance strategy works well for the 0/1 knapsack problem because accepting worse solutions doesn't improve the solution quality.
(iteratedLocalSearch.java)

```
if (candidate.fitness(inst) > global_best.fitness(inst)) {
    global_best = candidate.copy();
    no_improve_cnt = 0;
} else {
    no_improve_cnt++;
}
```

2(b) Environment & Setup

The ILS uses the following implementation:

Parameter	Value	Reason
Stopping condition	500 iteration without improvement	Balances runtime; found sufficient for all instances
Perturbation Strength	4 (2 removes + 2 adds)	Large enough to escape local optima but small enough to preserve solution quality
Local Search	1-opt + 2-opt	2-opt to reach optima when 1-opt cannot be reached
Acceptance Criterion	Strictly better only	Preserves best found solution; effective on knapsack instances
Initial Solution	Greedy using value-weight ratio and seeded noise	Provides a high quality valid starting solution
Repair	Remove lowest value-weight ratio first	Maximises new value when restoring feasibility
Random seed	Passed in from main	Ensure reproducibility of all results

Experimental Setup

All experiments were executed using a random seed provided by the user to ensure the results could be reproduced.The seed was also used for both GA and ILS on each instance to allow fair comparison. The seed controls the randomness in both initial solution and the perturbation ensuring reproducible results. Both meta-heuristic algorithms were run on all 11 problem instances. For the statistical analysis, both algorithms were run across 30 different seeds to generate paired observations for the Wilcoxon Signed-Rank Test. Runtime was measured in seconds using nanoTime and converted to seconds for reporting.

3. Results

The table below shows the best solution achieved by each algorithm across all 11 knapsack instances using the value seed 345

Problem Instance	Algorithm	Seed	Best Solution	Known Optimum	Runtime (s)
f1_l-d_kp_10_269	GA	345	295	295	0.0314
f1_l-d_kp_10_269	ILS	345	295	295	0.0015
f2_l-d_kp_20_878	GA	345	1024	1024	0.0449
f2_l-d_kp_20_878	ILS	345	1024	1024	0.0028
f3_l-d_kp_4_20	GA	345	35	35	0.045
f3_l-d_kp_4_20	ILS	345	35	35	0.0003
f4_l-d_kp_4_11	GA	345	23	23	0.0157
f4_l-d_kp_4_11	ILS	345	23	23	0.0005
f5_l-d_kp_15_375	GA	345	481068	481000	0.0402
f5_l-d_kp_15_375	ILS	345	481068	481000	0.0022
f6_l-d_kp_10_60	GA	345	52	52	0.0314
f6_l-d_kp_10_60	ILS	345	52	52	0.001
f7_l-d_kp_7_50	GA	345	107	107	0.0242
f7_l-d_kp_7_50	ILS	345	107	107	0.0007

f8_l-d_kp_23_10000	GA	345	9767	9767	0.0579
f8_l-d_kp_23_10000	ILS	345	9767	9767	0.0028
f9_l-d_kp_5_80	GA	345	130	130	0.0151
f9_l-d_kp_5_80	ILS	345	130	130	0.0003
f10_l-d_kp_20_879	GA	345	1025	1025	0.0421
f10_l-d_kp_20_879	ILS	345	1025	1025	0.0028
knapPI_1_100_1000_1	GA	345	9125.2	9125.2	0.1723
knapPI_1_100_1000_1	ILS	345	9147	9125.2	0.0123

Note: Since the values in f5 are decimals, they were scaled by a value of 1000 to maintain the integer values throughout the implementation.
The best solution of 481068 corresponds to 481.068 in the original problem. The listed known optimum of 481 scaled to 481000, and both algorithms exceeded this, which is consistent with the known optimum being a rounded lower bound

4. Statistical Analysis

To determine whether there is a statistically significant difference in performance between GA and ILS, a one-tailed Wilcoxon Signed-Rank Test was conducted at the 5% significance level (Wilcoxon, 1945). Both algorithms were run across 30 different seeds on all 11 instances, producing 330 paired observations per test.

The null hypothesis for both tests is that the two algorithms perform equivalently. The alternative hypothesis is one-tailed in each case.

Test 1: Solution Quality

H0: GA and ILS find equivalent solution values
H1: ILS finds better solution values than GA

Metric	Value
Total pairs	330
Non-zero differences	3
W+ (ILS better)	6.0
W− (GA better)	0.0
z-score	1.6036
z-critical (α = 0.05, one-tailed)	1.645

CONCLUSION:
We do not reject H0, since $z = 1.6036 \leq 1.645$.
At the 5% significance level, the difference in solution quality between GA and ILS is not statistically significant.
It should be noted that with only 3 non-zero differences out of 330 pairs, the sample of differing pairs is very small ($m < 10$), meaning the normal approximation used here is unreliable. An exact Wilcoxon table would be required for a precise p-value in this case.

Test 2: Runtime

H0: ILS and GA have equivalent runtimes
H1: ILS is quicker than GA

Metric	Value
Total pairs	330
Non-zero differences	330
W+ (ILS better)	54615.0
W− (GA better)	0.0
z-score	15.7440
z-critical (α = 0.05, one-tailed)	1.645

CONCLUSION:
We reject H0. $z = 15.7440 >> 1.645$.
ILS is significantly quicker than GA at the 5% significance level. The W− value of 0 indicates

that ILS was quicker than GA in every single one of the 330 paired runs without a single exception.

5. Critical Analysis

Solution Quality
ILS and GA achieved the known optimum on 10 out of 11 instances across all 30 seeds. This is a strong result, however it primarily reflects the nature of the benchmark instances rather than the sophistication of the algorithms. The majority of instances are small, ranging from 4 to 23 items, and both algorithms converge to the global optimum trivially on these. The only instance that revealed a meaningful quality difference was knapPI_1_100_1000_1 with 100 items, where the GA failed to reach the optimum on 3 out of 30 seeds, with a mean best value of 9125.2 vs ILS mean of 9147.0. This suggests that as problem size increases, the GA's random initialisation and crossover operator led to premature convergence on suboptimal solutions, while ILS's greedy construction and deterministic local search consistently finds the optimum even on the largest instance tested.

The Wilcoxon test on solution quality failed to reject H0 ($z = 1.6036$, $p > 0.05$). With only 3 non-zero differences out of 330 pairs, the test has very low statistical power. The near-miss z-score of 1.6036 against a critical value of 1.645 suggests that with more varied or harder instances, a significant quality difference would likely emerge.

Runtime
The runtime comparison produced a decisive result. ILS was significantly faster than GA in every single run ($z = 15.74$, $W^- = 0$, $p < 0.05$). The GA maintains a population of 100 individuals and runs up to 500 generations, performing up to 50,000 solution evaluations per run. ILS maintains a single solution and terminates after 500 non-improving iterations, requiring far fewer evaluations on the small instances tested. On the largest instance knapPI_1_100_1000_1, GA averaged **0.172s**, whereas ILS at **0.012s**, approximately 14x runtime difference.

Conclusion
For the provided instances, ILS is the more efficient algorithm. It achieves the same solution quality as GA but is significantly faster. However, the small size of the instances limits the generalisability of this conclusion. A more comprehensive evaluation on larger instances would be needed to determine which algorithm is superior in practice for real-world knapsack problems.

References:

Wilcoxon, F. (1945). Individual comparisons by ranking methods. Biometrics Bulletin, 1(6), 80-83.

Khuri, S., Bäck, T. and Heitkötter, J. (1994). The zero/one multiple knapsack problem and genetic algorithms. In Proceedings of the 1994 ACM Symposium on Applied Computing, pp. 188-193.

Nyathi, T. (2025). Artificial Intelligence COS314 Lecture Notes. Department of Computer Science, University of Pretoria. pp. 41 & 44-46.